

Car Rental – Booking Site and Fleet Management

A rental company's customer booking site and back office: availability, pricing, fleet calendar, check-out and check-in

IN ONE SENTENCE

Customers book cars online with live prices and availability; the desk runs the fleet, and no car is ever sold twice.

BUILT WITH

React 19, TypeScript, Vite; production: Node 22, Fastify 5, PostgreSQL (btree_gist), Docker, SMTP

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	7
5. Build it from scratch, step by step	9
6. Testing, deployment and known limits	13

1. What this project is

A car rental business lives on four questions: **can this car be booked, what should it cost, where is every car, and how hard is the fleet working?** This project answers them with a customer booking site and a staff back office in one app.

Customers search by pick-up and return location and time, see live prices and how many cars are left per class, add extras and book. Staff see a fleet calendar, check cars out and in (odometer, fuel, late fees), move bookings between cars, schedule maintenance, set prices and read a utilisation dashboard.

The repository holds **two versions**: the root folder is a client-side demo, and [car-rental-fleet-production-grade](#) runs on a server with PostgreSQL so online customers and several desks share the same fleet, with the database itself refusing a double booking.

At a glance	
Type	Public booking site + staff back office
Users	Renters, rental desk agents, owner/admin
Stack	React 19, TypeScript, Vite; production: Node 22, Fastify 5, PostgreSQL (btree_gist), Docker, SMTP
Demo data	28 cars in 5 classes at 2 locations; 120 days of history and 30 days of forward bookings (~60% utilisation)

Who uses it

User	What they do with it
Renter	Searches, compares classes and prices, books with extras, manages or cancels the booking.
Desk agent	Takes counter bookings, checks cars out and in, moves bookings, marks no-shows, uses the calendar.
Owner / admin	Sets prices, discounts, fees and rules, manages cars and users, reads the dashboard and activity log.

Two versions in this repository

	Demo version (root folder)	Production version (---production-grade folder)
Data	Browser localStorage	PostgreSQL shared by site and desks
Double booking	Checked in the browser	Checked on the server with the class's cars locked, and refused by a database exclusion constraint
Customers	Could see every booking	Only their own, by reference and email (same answer for either being wrong)
Times	Visitor's device time zone	The desk's time zone for everyone, correct across daylight-saving changes
Staff	Anyone who opens /admin	Logins: agent and admin
Emails	—	Confirmation and cancellation to customers; optional alert to the desk
Deployment	Static files	Docker image, compose file, health checks, CI with a database

2. The business problem

Small and mid-size rental companies often manage bookings in a spreadsheet or a whiteboard calendar. Two agents give the last car to two customers, one-way rentals leave cars stranded at the wrong branch, late returns and fuel are charged inconsistently, and nobody knows the fleet's utilisation — the number that decides whether to buy or sell cars.

The client wants **online booking with trustworthy availability**, a desk tool that handles the real-world details (cleaning time, one-way trips, overdue cars, maintenance), and a clear utilisation picture.

Pain point today	What it costs the business	How this project solves it
Double bookings of the last car	Angry customers at the counter	Availability engine plus (in production) server locks and a database exclusion constraint
Cars stranded after one-way rentals	Lost bookings at the other branch	Availability follows each car's location through one-way rentals
Inconsistent late and fuel charges	Disputes and lost revenue	Final bill previewed at check-in: late days after a grace period, fuel per eighth, other charges
No online booking	Customers book elsewhere	Public site with live prices, extras, age rules and booking references
Utilisation unknown	Fleet too big or too small	Dashboard: utilisation, revenue, average daily rate, rental length, per class

Value to the business

- More bookings online with no double-booking risk.
- Consistent charging at return.
- Fleet decisions based on measured utilisation.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Customer site	Search by pick-up/return location and time; each class shows the live price, cars left ("Only 1 left") or "Sold out".
FR-02	Customer site	Book with extras (zero-excess cover, extra driver, child seat, sat nav, with per-rental caps); driver details validated; minimum age and young-driver fee checked against the pick-up date; booking reference like <code>RC-7KQ2MX</code> .
FR-03	Customer site	Manage booking with reference plus email; free cancellation until 48 h before pick-up, then one day charged.
FR-04	Desk	Dashboard: utilisation (rented ÷ available car-hours, excluding servicing), revenue, average daily rate, average rental length, cancellation/no-show rate, cars on rent, overdue returns, cars ready per location, today's pick-ups and returns.
FR-05	Desk	Fleet calendar timeline with a "now" line; overdue cars in red; click to open the booking.
FR-06	Desk	Check-out and check-in with odometer (never backwards) and fuel; final bill preview before confirming.
FR-07	Desk	Move a booking to another genuinely free car; mark no-shows; cancel.
FR-08	Desk	Fleet: add/edit cars, current location, 30-day utilisation per car, maintenance blocks (refused if they clash with a booking).
FR-09	Desk	Prices & settings: daily rates and deposits per class, tax, 7+ and 28+ day discounts, fees, cleaning buffer, grace period, age rules, currency, locale.

Non-functional requirements

ID	Quality	Requirement
NFR-01	Correctness	No car is ever double-booked: checked at quote time, again at the moment of booking, and (production) enforced by the database.
NFR-02	Correctness	Money in whole cents; days follow the desk's calendar and time zone.
NFR-03	Security (production)	Passwords hashed with scrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.
NFR-04	Security (production)	Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https).
NFR-05	Security (production)	Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.
NFR-06	Security (production)	An activity (audit) log of logins and every business action, viewable by admins.
NFR-07	Security (production)	Customers can only find their own booking (reference and email, identical error either way); rate limits on booking (5 per 10 minutes per IP) and lookup; a honeypot for bots; plain-text emails.

ID	Quality	Requirement
NFR-08	Operations (production)	Runs as one Docker image next to PostgreSQL; health checks <code>/healthz</code> and <code>/readyz</code> ; database migrations run on start-up safely with several instances.
NFR-09	Operations (production)	Structured JSON logs with a request id (returned in the <code>x-request-id</code> header); cookies and tokens are redacted.

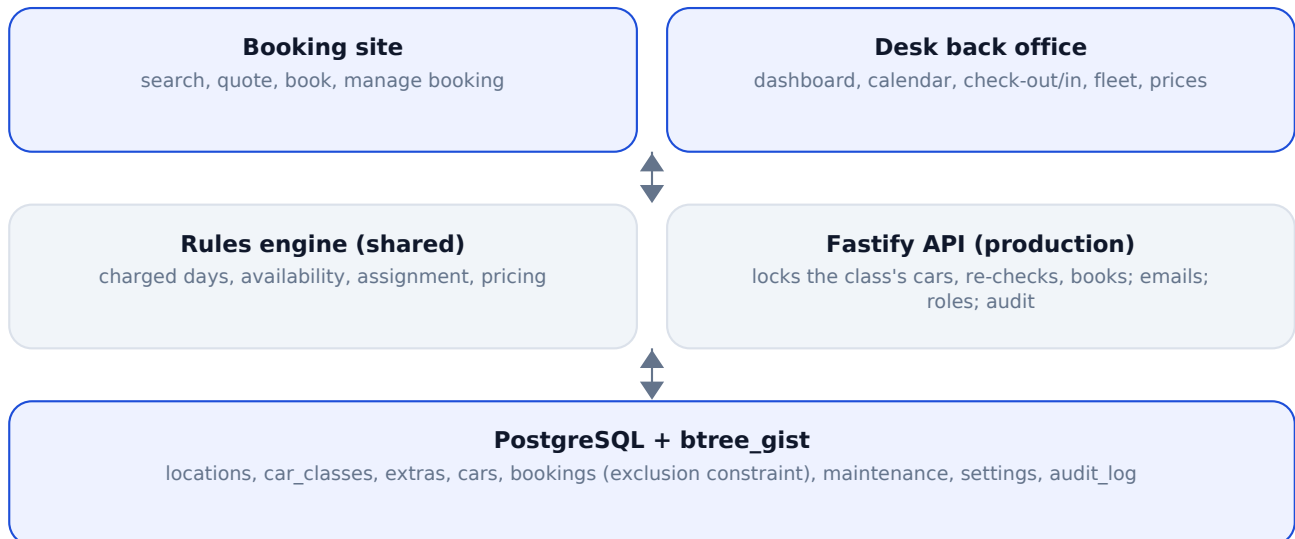
Out of scope for this release

- Card payments and deposit pre-authorisation (paid at the counter).
- Opening hours enforcement.
- SMS notifications.

4. How the solution is designed

All booking rules are pure functions in `src/lib/rental.ts` (demo) and `shared/` (production), so the site, the desk and the server use exactly the same logic. The server adds locking and a database constraint as the final guarantee.

Architecture



Technology choices

Part	Choice	Why
Web	React 19, TypeScript, Vite	One app for customers and staff; phone layout and dark mode.
Rules	Pure TypeScript functions	Unit-tested; shared by browser and server.
Server (prod)	Node 22 + Fastify 5	Transactions and row locks for the booking moment; SMTP emails.
Database (prod)	PostgreSQL with <code>btree_gist</code>	An exclusion constraint makes overlapping bookings of one car impossible even if code is wrong.
Tests	Vitest, Playwright	Concurrency tests, time-zone tests, browser journeys.

Data model

Table / file	What it holds
locations / car_classes / extras	Branches; classes with daily rate and deposit; extras with price and caps.
cars	Plate (unique ignoring spaces and case), class, current location, odometer, status.
bookings	Reference, customer, class and car, pick-up/return times and locations, extras, price breakdown, status, check-out/in readings, charges.
maintenance	Blocks of time a car is unavailable.
settings / users / sessions / audit_log	Business rules and prices; staff; sessions; activity log (production).

Key business rules

- **Charged days:** every started 24 h counts after a grace period (default 59 min), minimum 1 day.
- **Availability:** a car can take a rental only if there is no overlapping rental (including a cleaning buffer on both sides), no maintenance in the period, it will be at the pick-up location (following one-way rentals), a one-way return won't strand it away from its next booking, and an overdue car is treated as still out.
- **Assignment:** the free car idle the longest is chosen, spreading wear across the fleet.
- **Price:** rate × days, long-rental discount, capped extras, young-driver fee, one-way fee, then tax.
- **Database guarantee (production):** each booking stores `occupied` (pick-up to return plus the cleaning buffer); `EXCLUDE USING gist (car_id WITH =, occupied WITH &&)` makes two overlapping bookings of one car impossible. Statuses: Reserved, Active, Returned, Cancelled, No-show.

Folder layout

```
Root (demo):  src/lib/rental.ts (rules engine) · stats.ts · store/ · pages/ · data/ (generator)
car-rental-fleet-production-grade/:
    shared/ (rules) · server/ (Fastify, migrations, tests) · web/ · docs/ ·
    Dockerfile
```


5. Build it from scratch, step by step

Part A (steps 1–6) builds the rules engine and the demo app; Part B (steps 7–13) makes it safe for real bookings.

Before you start

- Node.js 22+; for Part B, PostgreSQL 14+ with the standard `btree_gist` extension, and Docker.

Step 1. Part A · Scaffold and model

```
npm create vite@latest car-rental-fleet -- --template react-ts
cd car-rental-fleet && npm i react-router-dom && npm i -D vitest playwright-core
```

1.1 Types for Location, CarClass, Car, Booking, Maintenance, Settings; money in cents.

Done when: The app runs with routing for `/` and `/#/admin`.

Step 2. Part A · The rules engine

- 2.1 Write `chargedDays`, `isCarFree`, `carLocationAt` (follows one-way rentals), `assignCar` (longest idle) and `quote` (discounts, extras caps, age fee, one-way fee, tax) in `src/lib/rental.ts`.
- 2.2 Unit-test every rule, especially the edge cases: grace period boundary, cleaning buffer, overdue cars, a one-way return that would strand the car.

Done when: Rules tests pass.

Step 3. Part A · Demo data

- 3.1 Seeded generator: 28 cars, 5 classes, 2 locations, 120 days of history and 30 days ahead at about 60% utilisation; a test proves no car is double-booked.

Done when: Generator test passes.

Step 4. Part A · Customer site

- 4.1 Search form, results per class with price and cars left, booking form with extras and validation, confirmation with reference, manage/cancel booking.

Done when: A booking made on the site appears on the desk calendar.

Step 5. Part A · Desk back office

- 5.1 Dashboard with utilisation and today's movements; fleet calendar timeline; check-out/in with final bill preview; move booking; fleet and maintenance; prices & settings.

Done when: A late return with missing fuel produces the expected bill.

Step 6. Part A · Tests and publishing

6.1 Playwright smoke test of customer and desk journeys on desktop, phone and dark mode; Pages workflow.

Done when: Tests pass; the demo is published.

Step 7. Part B · Set up the workspace

One repository with a server, a web app and shared code used by both.

7.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir car-rental-fleet-production-grade && cd car-rental-fleet-production-grade && git init
npm init -y # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg
zod nodemailer
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

7.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

Done when: `npm run typecheck` passes on the empty project.

Step 8. Part B · Configuration, database connection and migrations

8.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`, `SMTP_URL`) and refuse to start with a clear message when one is wrong.

8.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.

8.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

Done when: Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

Step 9. Part B · Schema with a double-booking guarantee

9.1 Tables as in the data model; plates unique on `upper(regexp_replace(plate, '\s', '', 'g'))`.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
-- occupied = the time the car is taken, including the cleaning buffer; NULL once cancelled /
no-show
occupied tstzrange,
CONSTRAINT no_double_booking EXCLUDE USING gist (car_id WITH =, occupied WITH &&) WHERE (
  occupied IS NOT NULL)
```

9.2 Move the rules engine into `shared/` unchanged so browser quotes and server bookings agree.

Done when: Inserting an overlapping booking directly in SQL fails.

Step 10. Part B · Logins, roles and request security

The same proven foundation every production app here uses.

- 10.1 **Passwords:** hash with Node's built-in `scrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 10.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 10.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 10.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 10.5 **Roles:** agent (bookings, check-out/in, fleet, calendar) and admin (also prices, settings, cars, users, activity log). Public routes for search, quote, book, lookup and cancel use rate limits, a honeypot and the reference-plus-email rule. A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 10.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 10.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

Done when: API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

Step 11. Part B · Booking API and emails

- 11.1 Book in one transaction: lock the class's cars (`SELECT ... FOR UPDATE`), recompute availability and price on the server, assign a car, insert; on conflict answer "just booked, please choose again".
- 11.2 Desk routes for check-out/in, move, no-show, cancel, maintenance, prices; server-side search and paging of bookings.
- 11.3 Emails (plain text) for confirmation and cancellation; optional desk alert.
- 11.4 All times interpreted in the desk's time zone (`TIMEZONE`), including in the customer's browser.

Done when: Concurrency test: a rush of simultaneous bookings for one class succeeds exactly as many times as there are free cars.

Step 12. Part B · Test it end to end

- 12.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 12.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 12.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then run a customer booking and cancellation, a counter booking through check-out and a late return, with the desk in New York and the browser in Karachi to

prove time handling, Fail on any script error, and save screenshots.

```
npm run typecheck && npm test  
npm run build && npm run smoke
```

Done when: All three suites pass locally.

Step 13. Part B · Package, automate and document

- 13.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 13.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 13.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.
- 13.4 `README.md` for users and `docs/OPERATIONS.md` for whoever runs it (https, backups, monitoring, upgrades).

Done when: CI is green on GitHub and `docker compose up -d --build` gives a working app.

6. Testing, deployment and known limits

How it is tested

- **Demo:** rules engine, store actions and demo-data integrity unit tests; browser smoke test of customer and staff journeys, phone and dark mode.
- **Production:** simultaneous bookings never overbook; the database refuses overlaps; bookings can't move onto busy cars; customer lookup/cancellation rules; under-age and missing terms refused; odometer can't go backwards; late return charges; role permissions; lockout; price changes reach new quotes but not existing bookings; plates unique ignoring spaces/case; time-zone handling across daylight-saving changes.

```
# demo
npm test && npm run build && npm run smoke
# production
npm run typecheck && npm test && npm run build && npm run smoke
```

Deploying and running it

- **Demo:** static hosting (GitHub Pages workflow runs when started by hand; or Netlify/Vercel/Cloudflare Pages).

```
cd car-rental-fleet-production-grade
cp .env.example .env # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE, SMTP_URL
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email owner@yourrentals.com --name "Owner Name"
# demo: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js (admin@ / agent@demo.local)
```

Known limits

- No online payments; deposits taken at the counter.
- Opening hours shown but not enforced.
- Prices and settings saved as separate requests (saving again finishes a partial save).
- Rate limits per server instance.

Sensible next steps

- Stripe payments with deposit pre-authorisation.
- SMS reminders before pick-up.
- Dynamic pricing by demand and season.